

Project 4 Report

Pawat Suntharanund

Architectural Design:

Mutual authentication is achieved using TLS with certificates. The client authenticates the server by verifying the server certificate against the trusted CA. The server authenticates the client through the client certificate present during the TLS handshake. After secure channel is established, the user performs login by signing the login statement with their RSA private key. The server verifies this signature using the corresponding stored public key before issuing a session token.

For my implementation, I included the following the cryptography library from the requirements document as well as the “secrets” and “time” modules from the python standard library. From the cryptography library, I used the following:

- 1) RSA for digital signatures during login
- 2) AES-GCM to encrypt confidential documents per the requirements of the project
- 3) RSA-OAEP with SHA-256 to encrypt a randomly generated AES key before storage
- 4) RSA PKCS1v15 to digitally sign integrity-protected documents
- 5) SHA-256 for the hashing algorithm used for both signatures and OAEP

The confidential documents were stored as encrypted data together with an encrypted AES key and nonce. Integrity protected documents were stored together with their respective digital signatures.

In terms of the user information and document metadata, the way that I stored them was by using active and user sessions that were stored in server memory using dictionaries that would map session tokens to authenticated users.

Each document has an associated metadata file stored as JSON which contains the following:

- 1) Document ID
- 2) Security Flag
- 3) Document Owner
- 4) Current grant access (if applicable)
 - this includes target user, access right, expiration time

Implementation:

Login:

The client prompts for the user ID and private key, generates the login statement, signs with the RSA private key, encodes the signature and sends it to the server. The server will then validate the statement, signature and key. It will also invalidate any previous sessions for the user and generates a secure session token using `secrets.token_urlsafe()`.

Checkin:

The client will read the file, encode the content and sends it with the requested security flag.

If the security flag is 1, the server will first generate a random AES key. Then, it will encrypt the document via AESGCM. Next it will encrypt the AES key using the RSA public key and finally stores the encrypted document, AES key, nonce and metadata in the checkin folder.

If the security flag is 2, the server will sign the document using its RSA private key and stores both the document and signature in the checkin folder. Ownership and grant information are preserved.

Checkout:

The client will request a file using the document ID. The server will first validate the user's session, then verify the ownership of the file or grant authorization to the file. Next, it will check whether any grant has expired. Then, depending on the security flag it will either decrypt the file(s) before transmission(1) or verify the signatures before returning the file(s)(2). The returned document is finally encoded and stored by the client in the checkout folder.

Grant:

Only the owner of a file may grant permissions, and each grant specifies the target user (user ID 0 for all users), the access type(check in, checkout or both) as well as the expiration time. The newest grant will replace any previous grant for that document. Authorization checks verify ownership, the recipient, the requested operation and the expiration time before allowing any access.

Delete:

Only the owner of a file may delete the file. For confidential documents, the encrypted AES key is destroyed before deleting the encrypted document and its' related files (metadata, nonce). Destroying the encryption key helps provide cryptographic deletion because encrypted data can no longer be decrypted even if it were recovered from storage. Integrity documents have document, signature and metadata removed.

Logout:

During logout, every document currently in the checkout folder is moved to the check in folder and uploaded back to the server using integrity flag. If any of the uploads fail, the logout is aborted and the session remains active. After all the uploads complete successfully, the client requests logout and the server will invalidate the session token.

Static Code Analysis:

As I developed within my own virtual environment instead of the provided environment, I used the python LSP in Neovim to check for any syntax errors.

For validating filenames, I used `os.path.basename()` and rejecting absolute paths. For all file and cryptographic operations I made sure to have exception handling. To ensure secure session tokens I used `secrets.token_urlsafe()` as it would help ensure that there would not be a realistic chance of an attacker figuring out an active session's token.

Threat Modeling:

Currently, my implementation handles the following threats:

Unauthorized access:

Only authenticated users with valid certificates and RSA login signatures will be able to receive session tokens. Authorization is enforced through document ownership and temporary grants.

Eavesdropping:

TLS encrypts all communication between the client and the server. Confidential documents are additionally encrypted using AES before being stored on the server.

Document tampering:

Files with security flag of two(integrity) are digitally signed. Before checkout, the server will verify the signature and rejects any modified files.

Path traversal:

All document IDs and filenames are validated using `os.path.basename()` on top of rejecting absolute paths. This helps to prevent directory traversal attacks.

Session hijacking:

I used the 'secrets' module in order to ensure that it is very difficult for attackers to figure out session tokens. Previous sessions for the same user are invalidated during login as a further protection mechanism.

Unauthorized document deletion:

Only the original owner of a file can delete it.

Confidential data recovery:

For confidential files, we delete the encrypted AES key as well so as to prevent future decryption, which provides cryptographic deletion.

Some threats that I would need more time to implement would be protection against the server's private key being compromised, client private keys being compromised, DDoS attacks, and people within the system with admin rights that would be malicious actors.